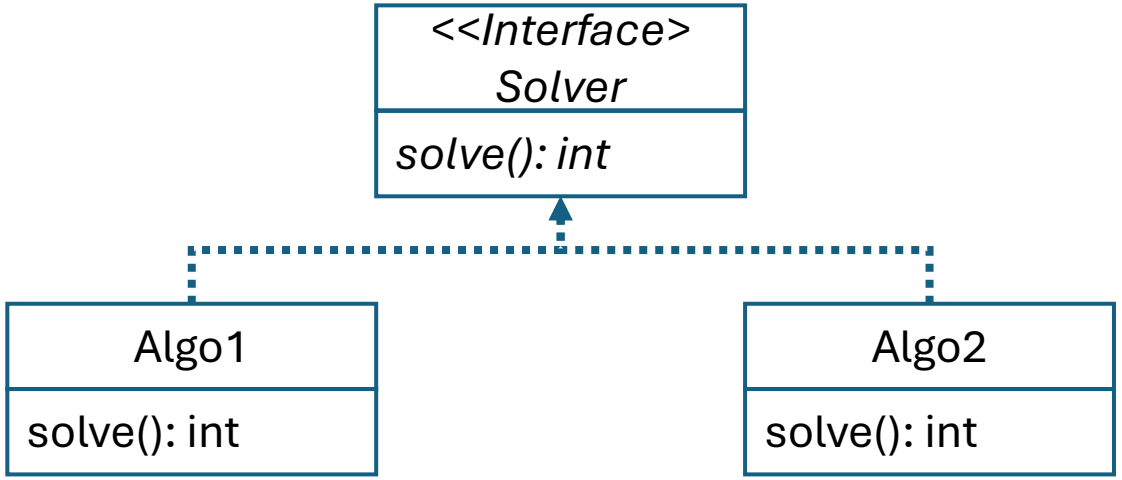
Lab02 Benchmarking

- Implement three prime-checking algorithms: isPrime0, isPrime1, and isPrime2.
  - Verify correctness by counting prime numbers below 100; the expected result is 25 primes.
- Benchmark each implementation for input sizes from 100,000 to 1,000,000.
  - Record the number of primes found and runtime in milliseconds for each algorithm.
- Analyze how the runtime of isPrime0 grows as input size increases.
- Calculate time ratio and increased factor to study runtime growth trends.
- Compare runtime behavior across different machines using ratios instead of raw time.
- Plot runtime comparison graphs: isPrime0 vs. isPrime1, and isPrime1 vs. isPrime2.
- Submit the final PDF report together with IsPrime0.java, IsPrime1.java, and IsPrime2.java.

OOP: Abstraction (Interface)

- An interface defines what a class must be able to do.
- Different classes can implement the same interface methods in different ways.




```
interface Solver {
    public int solve();
}
class Algo1 implements Solver {
    public int solve() {
        println("way1"); return 0;
    }
}
class Algo2 implements Solver {
    public int solve() {
        println("way2"); return 0;
    }
}

Solver[] engines = {new Algo1(), new
Algo2()};
for (var engine : engines)  {
    engine.solve();
}
```